

Topic: Interrupt Structure and its Servicing — Self-Learning

Q. Discuss the Interrupt structure of the 8086 microprocessor. Differentiate between Hardware and Software interrupts, and explain the sequence of steps the CPU follows to service an interrupt.

 **Definition to Remember**

An **Interrupt** pauses the CPU's normal execution to handle an urgent task (an Interrupt Service Routine or ISR). The 8086 supports 256 interrupts. It uses an **Interrupt Vector Table (IVT)** in lower memory to locate the ISR addresses, ensuring the CPU state is saved on the stack before jumping.

Interrupt Structure & Servicing in 8086 (Self-Learning)

Type	Source	Characteristics	Examples
Hardware	External physical devices	Triggered by electrical signals to CPU pins. Can be Maskable (ignored if IF=0) or Non-Maskable.	INTR (Maskable), NMI (Non-Maskable).
Software	Internal program execution	Triggered by the INT n instruction or internal CPU exceptions. Cannot be masked.	INT 21H (DOS), Divide-by-zero (Type 0).

2. The Interrupt Vector Table (IVT)

- Located at absolute memory `00000H` to `003FFH` .
- Supports 256 interrupt types (Type 0 to 255).
- Each type gets exactly **4 bytes** in the IVT (2 bytes for the ISR's `IP` offset, 2 bytes for its `CS` segment).

3. Interrupt Servicing Sequence

When a valid interrupt is acknowledged, the 8086 performs the following strict sequence to ensure it doesn't lose its place in the main program:

1. **Push Flags:** Pushes the 16-bit Flag Register onto the Stack.
2. **Clear Flags:** Clears the Interrupt Flag (IF=0) and Trap Flag (TF=0) to prevent other maskable interrupts from interrupting the ISR.
3. **Save Context:** Pushes the current Code Segment (CS) and Instruction Pointer (IP) onto the Stack (saves the return address).
4. **Fetch ISR Address:** Multiplies the interrupt type number by 4 to find its IVT entry. Loads the new CS and IP values.
5. **Execute ISR:** Jumps to and executes the Interrupt Service Routine.
6. **Return (`IRET`):** The ISR ends with the `IRET` instruction, which pops the IP, CS, and Flags back from the stack, perfectly resuming the main program.

★ Must-Write Points (for 10 marks)

1. An **Interrupt** forces the CPU to pause execution and run a specific routine (ISR).
2. **Hardware Interrupts**: External signals. INTR is maskable; NMI is non-maskable (for critical errors).
3. **Software Interrupts**: Triggered by **INT n** instructions or exceptions (divide-by-zero).
4. **Interrupt Vector Table (IVT)**: Occupies the lowest 1KB of memory (00000H-003FFH).
5. IVT holds 256 entries. Each entry is 4 bytes (CS and IP for the ISR).
6. **Servicing Sequence**: Push Flags → Clear IF/TF → Push CS and IP → Load new CS/IP from IVT → Execute ISR.
7. The ISR must end with **IRET** to pop the IP, CS, and Flags from the stack and resume the program.

⚡ Quick Recall

Interrupts → Hardware (INTR/NMI) vs Software (INT n) → IVT (Lowest memory, 256 types, 4 bytes each for CS:IP) → Sequence: Push Flags → Clear IF/TF → Push CS:IP → Fetch ISR from IVT → Run ISR → IRET (Pop all back)